Regularização de sinistros: atrasos e valores pendentes por ramo e canal

Fundamentos de Programação — Trabalho individual, Proposta 10 · Mestrado em Business Analytics — Executive

Estudante: [nome e número] · Dados fictícios: dados_trabalho_10_seguros.csv

1. Contexto e dados

Uma seguradora quer acompanhar a regularização dos sinistros por ramo e por canal de participação, para identificar atrasos e valores pendentes e assim organizar o trabalho da equipa. A análise **apoia a priorização do acompanhamento**; não pretende automatizar nem influenciar decisões sobre sinistros.

O ficheiro contém 12 sinistros fictícios (S001 a S012), 3 em cada um dos 4 ramos (Automóvel, Habitação, Responsabilidade Civil e Viagem), distribuídos por 3 canais (Agente, Corretor e Digital), com 4 sinistros por canal. Cada linha tem o valor reclamado, o valor regularizado, os dias de resolução e o estado (Fechado ou Pendente). Há 7 sinistros fechados e 5 pendentes.

2. Preparação e verificação de qualidade

O CSV é lido com `Path` e `pd.read_csv()`. A verificação de qualidade corre **antes de qualquer conversão ou preenchimento**, para que tipos e valores em falta originais não fiquem escondidos:

Verificação	Resultado
Dimensão	12 linhas × 7 colunas
Tipos	valor_reclamado, valor_regularizado e dias_resolucao são inteiros; sinistro, ramo, canal e estado são texto
Valores em falta	0 em todas as colunas
Linhas duplicadas / IDs duplicados	0 / 0
Categorias	Estado: Fechado, Pendente · Ramos: 4 valores já coerentes com os limites

Como os dados estão limpos, não foi preciso excluir nem imputar nada. Mesmo assim, o código normaliza os nomes de ramo (por exemplo, "Automóvel" e "Responsabilidade Civil" passam a "Automovel" e "ResponsabilidadeCivil"), para que a comparação com os limites não falhe por diferenças de escrita, e converte as colunas numéricas com `pd.to_numeric`.

3. Decisões de implementação

Regra central e pseudocódigo. A regra mais importante é a que decide se um sinistro está fora de prazo. Foi desenhada em pseudocódigo antes de ser programada:

```
FUNÇÃO limite_regularizacao(ramo):
    normalizar o nome do ramo
    DEVOLVER limite em dias (Automovel 25, Habitacao 25,
             Viagem 10, ResponsabilidadeCivil 35)
             ou NADA se o ramo não existir

FUNÇÃO cumpre_prazo(ramo, dias_resolucao):
    limite <- limite_regularizacao(ramo)
    SE limite = NADA OU dias_resolucao em falta:
        DEVOLVER VERDADEIRO        (não assinalar atraso sem informação)
    DEVOLVER dias_resolucao <= limite

PARA CADA sinistro:
    fora_prazo <- NÃO cumpre_prazo(ramo, dias_resolucao)
```

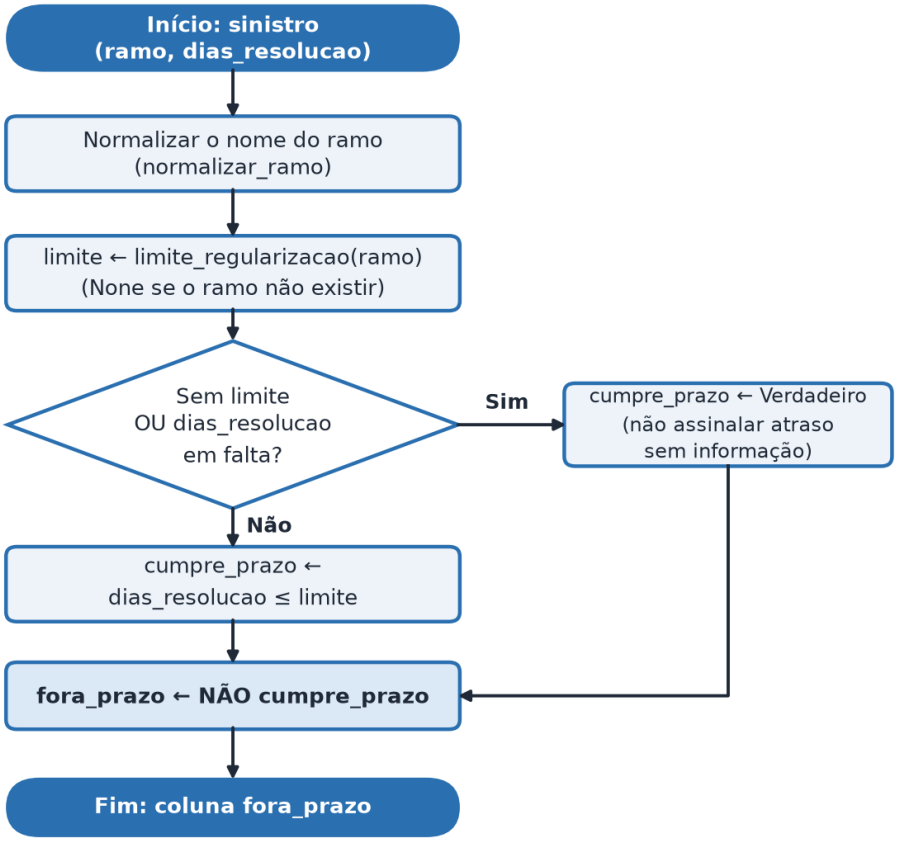


Figura 1 — Fluxograma da regra fora_prazo (equivalente ao pseudocódigo acima).

Colunas calculadas. `valor_pendente` é igual ao valor reclamado quando o estado é Pendente e zero nos restantes casos. `fora_prazo` é uma coluna lógica obtida com `apply` sobre `cumpre_prazo`, que por sua vez chama `limite_regularizacao`.

Pressuposto assumido. Sem limite conhecido ou sem dias de resolução, o sinistro não é assinalado como atrasado. Prefere-se não acusar atraso por falta de informação, o que é coerente com o objetivo de apoio. Nestes dados o pressuposto não se aplica, porque não há valores em falta nem ramos desconhecidos.

Classe `Sinistro`. Tem os atributos id_sinistro, ramo, estado e valor_reclamado e o método `valor_pendente()`, com a mesma regra da coluna. É demonstrada com o primeiro sinistro do

CSV (S001, Fechado), cujo valor pendente é 0 €.

Resumo e exportação. O resumo por ramo usa `groupby()` com `agg()`. Os prioritários são os sinistros pendentes *ou* fora de prazo, ordenados por valor pendente e por dias de resolução, ambos de forma decrescente, e exportados para `sinistros_prioritarios.csv` com `index=False`.

4. Resultados

4.1 Resumo por ramo

Ramo	N.º	Reclamado (€)	Regularizado (€)	Pendente (€)	Dias médios	% fora prazo
Automóvel	3	11 100	5 400	5 200	29,3	66,7%
Habitação	3	13 800	7 100	6 200	24,0	33,3%
Responsabilidade Civil	3	25 800	7 000	18 200	46,0	100,0%
Viagem	3	2 830	1 500	1 200	10,0	33,3%
Total	**12**	**53 530**	**21 000**	**30 800**	**27,3**	**58,3%**

Tabela 1 — Resumo por ramo. Dias médios e % fora de prazo calculados sobre todos os sinistros do ramo.

Dos 53 530 € reclamados, 30 800 € (57,5%) estão pendentes. Sete dos 12 sinistros (58,3%) excedem o limite do respetivo ramo, e **todos os 5 pendentes estão fora de prazo**.

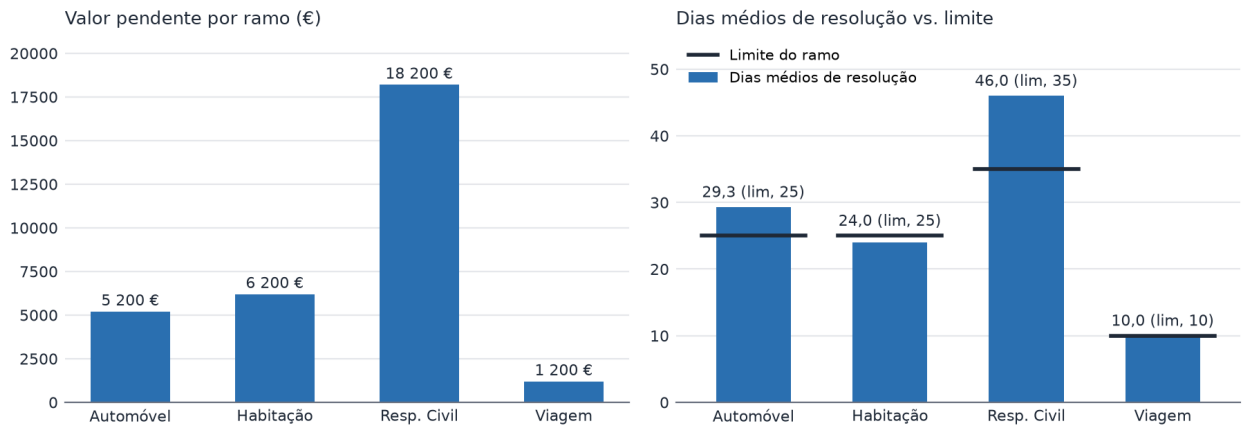


Figura 2 — Valor pendente por ramo e dias médios de resolução face ao limite de cada ramo.

4.2 Sinistros prioritários

O ficheiro `sinistros_prioritarios.csv` tem 7 processos: os 5 pendentes e 2 fechados fora de prazo. Como todos os pendentes já excedem o limite, neste conjunto o critério "pendente ou fora de prazo" coincide com "fora de prazo".

Sinistro	Ramo	Canal	Estado	Pendente (€)	Dias	Limite	Excesso
S012	Resp. Civil	Corretor	Pendente	9 800	55	35	+20
S004	Resp. Civil	Agente	Pendente	8 400	45	35	+10
S002	Habitação	Digital	Pendente	6 200	32	25	+7
S009	Automóvel	Corretor	Pendente	5 200	42	25	+17
S007	Viagem	Digital	Pendente	1 200	14	10	+4
S008	Resp. Civil	Agente	Fechado	0	38	35	+3
S005	Automóvel	Digital	Fechado	0	28	25	+3

Tabela 2 — Processos prioritários, pela ordem do ficheiro exportado (valor pendente e dias, decrescente).

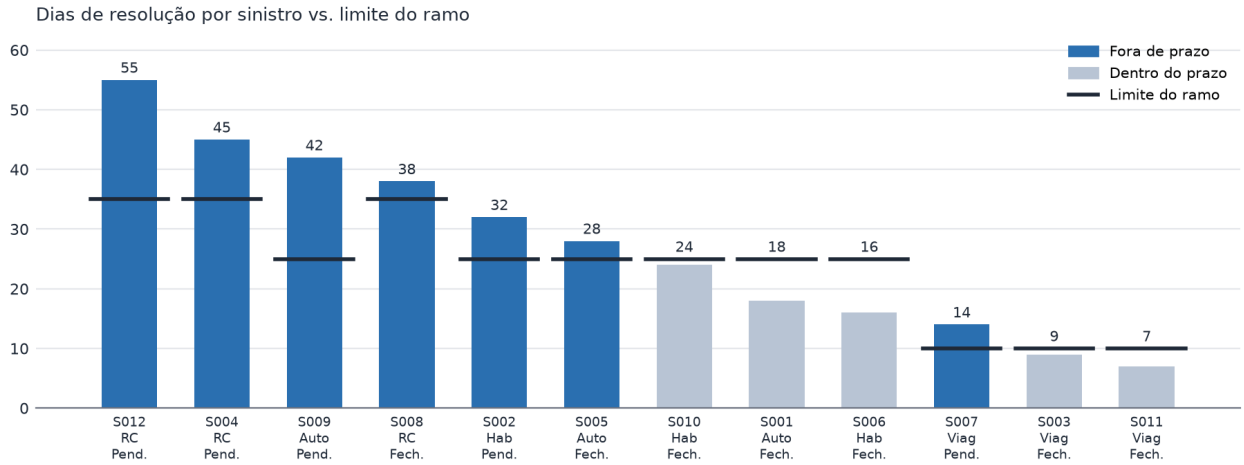


Figura 3 — Dias de resolução de cada sinistro face ao limite do respetivo ramo. RC = Responsabilidade Civil; Pend. = Pendente; Fech. = Fechado.

4.3 Canais de participação

Canal	Sinistros	Pendente (€)	Fora de prazo	Dias médios	Prioritários
Agente	4	8 400	2 (50%)	27,0	2 (28,6%)
Corretor	4	15 000	2 (50%)	30,5	2 (28,6%)
Digital	4	7 400	3 (75%)	24,5	3 (42,9%)

Tabela 3 — Indicadores por canal (cada canal tem 4 sinistros).

Canal	Automóvel	Habitação	Resp. Civil	Viagem	Total
Agente	0	0	2	0	2
Corretor	1	0	1	0	2
Digital	1	1	0	1	3
Total	**2**	**1**	**3**	**1**	**7**

Tabela 4 — Número de processos prioritários por canal e ramo (resultado de pd.crosstab).

5. Interpretação e resposta às questões

Que ramo necessita de maior atenção operacional? A **Responsabilidade Civil**. Os indicadores que o fundamentam são:

- **100% dos sinistros fora de prazo** (3 em 3), contra 66,7% no Automóvel e 33,3% em Habitação e Viagem.
- **Dias médios de 46,0** face a um limite de 35 (+11 dias), o maior desvio dos quatro ramos. O Automóvel tem 29,3 face a 25.
- **18 200 € pendentes**, 59,1% do total pendente, e 70,5% do que foi reclamado no ramo continua por regularizar (Automóvel: 46,8%; Habitação: 44,9%; Viagem: 42,4%).

O Automóvel merece a segunda atenção, sobretudo pelo S009 (42 dias, 17 acima do limite, 5 200 € pendentes). A Habitação e a Viagem cumprem em média o limite (24,0 face a 25 e 10,0 face a 10), mas cada uma tem um sinistro pendente fora de prazo (S002 e S007); a Viagem tem o limite mais curto e apenas 1 200 € pendentes.

Os prioritários concentram-se num canal ou num ramo? Há alguma concentração por **ramo**: a Responsabilidade Civil tem 3 dos 7 prioritários (43%), contra 2 do Automóvel e 1 de cada um dos outros dois. Por **canal** a concentração é fraca em número: o Digital tem 3 e Agente e Corretor têm 2 cada. O Corretor destaca-se em valor, com 15 000 € (48,7% do pendente) em apenas dois

processos (S012 e S009), e em dias médios (30,5). Com 4 sinistros por canal, isto reflete casos individuais e não prova um padrão do canal.

Que variáveis adicionais ajudariam a interpretar atrasos, sem atribuir responsabilidade a clientes ou equipas? Variáveis que descrevam o processo e não as pessoas:

- datas de cada fase (participação, documentação completa, peritagem, decisão, pagamento), separando o tempo de espera externa do tempo interno;
- complexidade do sinistro: danos corporais ou a terceiros, litígio, necessidade de resseguro, valor e número de intervenientes;
- existência de peritagem externa ou de pedidos de documentação, e quantas vezes o processo foi reaberto;
- carga de trabalho e capacidade no período, e sazonalidade (por exemplo, picos por eventos climáticos);
- a forma como o prazo é contado (dias úteis ou de calendário) e o seu historial para cada ramo, para avaliar se os limites são realistas.

6. Conclusões, limitações e recomendação

Conclusões. 58,3% dos sinistros estão fora de prazo e 57,5% do valor reclamado está pendente. A Responsabilidade Civil é o ramo mais crítico (100% fora de prazo, 46 dias em média e 59% do valor pendente). O Automóvel é o segundo e o Digital tem a maior proporção de sinistros fora de prazo (75%), embora com valores mais baixos.

Limitações. (i) A amostra é muito pequena (12 sinistros, 3 por ramo), por isso percentagens como 100% mudam muito com um só caso. (ii) Para os sinistros pendentes, `dias_resolucao` foi interpretado como o tempo em aberto até à data dos dados; se tiver outro significado, a classificação muda. (iii) Os limites são fixos por ramo e não consideram a complexidade dos casos. (iv) Não há datas nem informação sobre as causas dos atrasos, por isso a análise descreve onde estão os atrasos e não os explica.

Recomendação. Dar prioridade de acompanhamento aos dois processos pendentes de Responsabilidade Civil (S012 e S004, 18 200 €), seguidos do S009 (Automóvel) e do S002 (Habitação). Convém confirmar com a equipa se os 35 dias de Responsabilidade Civil são realistas para a complexidade típica do ramo. Recomenda-se ainda recolher as datas de cada fase e a complexidade para uma análise futura, e usar o resultado como lista de trabalho: a decisão sobre cada sinistro continua a ser da equipa.

Anexo — Código Python (relatorioFundamentosDeProgramacao.py)

```
"""Proposta 10 - Regularização de sinistros."""
from pathlib import Path

import pandas as pd

# Pseudocódigo da regra central (fora_prazo):
#   limite <- limite_regularizacao(ramo)
#   SE limite ou dias_resolucao em falta: considerar dentro do prazo
#   cumpre_prazo <- dias_resolucao <= limite
#   fora_prazo <- NÃO cumpre_prazo

COL_ID = "sinistro"
COL_RAMO = "ramo"
COL_CANAL = "canal"
COL_ESTADO = "estado"
COL_RECLAMADO = "valor_reclamado"
COL_REGULARIZADO = "valor_regularizado"
COL_DIAS = "dias_resolucao"

LIMITES = {
    "Automovel": 25,
    "Habitacao": 25,
    "Viagem": 10,
    "ResponsabilidadeCivil": 35,
}

MAPEAMENTO_RAMO = {
    "Automovel": "Automovel",
    "Automóvel": "Automovel",
    "Habitacao": "Habitacao",
    "Habitação": "Habitacao",
    "Viagem": "Viagem",
    "Responsabilidade Civil": "ResponsabilidadeCivil",
    "ResponsabilidadeCivil": "ResponsabilidadeCivil",
}


def normalizar_ramo(ramo):
    """Padroniza nomes de ramos para comparar com os limites."""
    if pd.isna(ramo):
        return ramo
    valor = str(ramo).strip()
    return MAPEAMENTO_RAMO.get(valor, valor)


def limite_regularizacao(ramo):
    """Devolve o limite em dias do ramo (None se desconhecido)."""
    return LIMITES.get(normalizar_ramo(ramo))


def cumpre_prazo(ramo, dias_resolucao):
    """True se os dias não excedem o limite do ramo.
    Pressuposto: sem limite ou sem dias, não se assinala atraso."""
    limite = limite_regularizacao(ramo)
    if limite is None or pd.isna(dias_resolucao):
        return True
    return dias_resolucao <= limite


class Sinistro:
    """Representa um sinistro e calcula o respetivo valor pendente."""

    def __init__(self, id_sinistro, ramo, estado, valor_reclamado):
        self.id_sinistro = id_sinistro
        self.ramo = ramo
        self.estado = estado
        self.valor_reclamado = valor_reclamado

    def valor_pendente(self):
        return self.valor_reclamado if self.estado == "Pendente" else 0


def main():
    # 1. Leitura e qualidade dos dados
    base_dir = Path(__file__).resolve().parent
    csv_file = base_dir / "dados_trabalho_10_seguros.csv"

    if not csv_file.exists():
        raise FileNotFoundError(
            f"Ficheiro de dados não encontrado: {csv_file}\n"
            "Verifique se o CSV está na mesma pasta do script."
        )

    df = pd.read_csv(csv_file)

    # Verificação de qualidade feita ANTES de qualquer conversão ou preenchimento,
```

```
    # para que valores em falta e tipos originais não fiquem escondidos.
    print("Dimensão:", df.shape)
    print(df.dtypes)
    print("Valores em falta por coluna:")
    print(df.isna().sum())
    print("Linhas duplicadas:", df.duplicated().sum())
    print("IDs de sinistro duplicados:", df[COL_ID].duplicated().sum())
    print("Estados:", df[COL_ESTADO].unique(), "| Ramos:", df[COL_RAMO].unique())

    # Preparação: normalizar ramos e garantir tipos numéricos
    df[COL_RAMO] = df[COL_RAMO].map(normalizar_ramo)
    df[COL_DIAS] = pd.to_numeric(df[COL_DIAS], errors="coerce")
    df[COL_RECLAMADO] = pd.to_numeric(df[COL_RECLAMADO], errors="coerce").fillna(0)
    df[COL_REGULARIZADO] = pd.to_numeric(df[COL_REGULARIZADO], errors="coerce").fillna(0)

    # 2. valor_pendente
    df["valor_pendente"] = df[COL_RECLAMADO].where(
        df[COL_ESTADO].astype(str).str.strip().eq("Pendente"), 0
    )

    # 3. fora_prazo
    df["fora_prazo"] = ~df.apply(
        lambda linha: cumpre_prazo(linha[COL_RAMO], linha[COL_DIAS]), axis=1
    )

    # 4. Resumo por ramo
    resumo = df.groupby(COL_RAMO).agg(
        n_sinistros=(COL_ID, "count"),
        valor_reclamado=(COL_RECLAMADO, "sum"),
        valor_regularizado=(COL_REGULARIZADO, "sum"),
        valor_pendente=("valor_pendente", "sum"),
        dias_medios=(COL_DIAS, "mean"),
        pct_fora_prazo=("fora_prazo", lambda s: s.mean() * 100),
    ).round(2)
    print(resumo)

    # 5. Demonstração da classe
    linha = df.iloc[0]
    sinistro = Sinistro(
        linha[COL_ID], linha[COL_RAMO], linha[COL_ESTADO], linha[COL_RECLAMADO]
    )
    print(sinistro.id_sinistro, sinistro.valor_pendente())

    # 6. Exportação dos prioritários
    prioritarios = df[(df[COL_ESTADO].astype(str).str.strip().eq("Pendente")) | df["fora_prazo"]]
    prioritarios = prioritarios.sort_values(
        by=["valor_pendente", COL_DIAS], ascending=[False, False]
    )
    prioritarios.to_csv(base_dir / "sinistros_prioritarios.csv", index=False)

    # Apoio às questões de análise
    print(df[COL_CANAL].value_counts(normalize=True))
    print(prioritarios[COL_CANAL].value_counts(normalize=True))
    print(pd.crosstab(prioritarios[COL_CANAL], prioritarios[COL_RAMO]))


if __name__ == "__main__":
    main()
```